

Project Post-Mortem: Scape

Kessel DIAROUMEYE

January 18, 2026

Phase 1: Initial Setup & Controls

The Git & Unity Struggle

Starting out, I hit a frustrating wall just trying to commit the empty Unity project. Even though I thought I had my `.gitignore` set up, I kept getting hit with "Filename too long" errors because of Windows path limits, and "Permission denied" errors because Unity was locking files in the background.

At first, I thought the only solution was to close the Unity Editor every single time I wanted to commit. I quickly realized this wasn't sustainable—reopening the project takes way too long, especially as the project grows.

It turned out my hypothesis about the `.gitignore` was half-right. The file was there, but it was in the parent folder instead of inside the actual Unity project folder (**Scape**). Git wasn't filtering the deep **Library** and **Temp** folders correctly. The fix was a mix of things: allowing long file paths in the Git config, but most importantly, moving the `.gitignore` into the correct subfolder. Once I did that, I could commit without closing Unity.

The QWERTY vs. AZERTY Mix-up

When I moved on to the controls, I ran into a weird issue where the game ignored my code. I programmed it for **ZQSD** (AZERTY), but Unity kept behaving like I was using a QWERTY keyboard (I had to press W to move forward).

I realized that Unity's Input System defaults to the physical location of keys on a US keyboard, regardless of what my script said. I stopped trying to manually select keys from the dropdown list and instead used the "**Listen**" feature in the Input Action window. By simply pressing my physical 'Z' key, Unity mapped the correct Key Code instantly. That sorted out the movement logic for good.

Phase 2: Core Mechanics & Workflow Stability

The Raycast vs. Trigger Pivot

When implementing the inventory system, I hit a snag with object detection. My initial plan was a "look-and-press" system using Raycasts, but the objects simply refused to be detected. I spent a significant amount of time adding visual debug lines and checking logs, but the raycast logic remained inconsistent.

Rather than stalling progress on a single mechanic, I decided to pivot. I switched from a Raycast system to a **Trigger-based system** (Physics Colliders). Now, the player simply walks over an object to collect it. This shift highlighted a crucial dependency: I couldn't verify if the pickup logic was actually working without visual feedback. This forced me to prioritize building the **UI** much earlier than planned, just to debug the gameplay loop.

The Compilation "Deadlock"

I learned a hard lesson about Unity's architecture: **You cannot code through red errors.**

I tried to ignore a compilation error to work on a different script, but Unity stops compiling the entire project the moment one script breaks. This creates a deadlock where you can't test *anything* until the specific error is resolved. It reinforced the importance of working in small steps and fixing errors immediately. This is where Git became invaluable—allowing me to revert to a "clean" state whenever I dug myself into a hole I couldn't fix quickly.

Git & Range Refinements

On a positive note, the Git file-locking issue from Phase 1 resolved itself. It appears the "close Unity to commit" requirement was a one-time bug related to the initial `.gitignore` setup; I can now commit freely while the editor is open.

Finally, I spent a lot of time tweaking interaction ranges. I found that if the range is too strict, the game feels unresponsive. The lesson here was to be generous with collision sizes and interaction distances to ensure the game feels good to play, rather than mathematically perfect.

Phase 3: Scope Management & Final Polish

The "Frankenstein" Asset Struggle

My original plan relied on collaborating with a friend skilled in 3D modeling, but due to unexpected changes, I had to proceed solo. This forced me to rely on free assets from the store, which created a massive consistency problem.

It was impossible to find visual continuity between the different packs; the game looked like a "Frankenstein" project, and worse, the scripts included in these assets conflicted with my own code. Due to the strict time constraints, I made the hard decision to **cut them entirely**. This taught me that relying on external assets requires as much integration time as building them from scratch.

The Ambition Trap (Pivoting to Escape)

I lost a significant amount of time early on trying to craft a complex story, deep mechanics, and a polished look. I realized too late that my initial concept was simply too ambitious for the timeframe.

To save the project, I had to drastically **scale down**. I abandoned the narrative heavy-lifting and pivoted to a straightforward **Escape Game**. This shift allowed me to focus on a singular, functional gameplay loop rather than a broken, sprawling RPG.

The "Missing Reference" Headache

In the final stretch, I ran into a persistent issue with Prefabs. Buttons and interactions I had previously set up suddenly stopped working because their references in the Inspector had turned to "Missing" (likely due to script changes or moving files).

Fixing this manually was confusing, so I adopted a new workflow: I cross-referenced my development blog/logs and fed the specific error context to an AI. By explaining the broken logic to the AI, I was able to quickly identify exactly which links had broken in the Inspector and re-assign the correct scripts without rewriting the whole system.